

Servir plusieurs sites web avec le même serveur

Note : Pour pouvoir faire ce TP, il faut avoir fait les TP « héberger une page web », « enregistrer un nom de domaine pour votre conteneur » et « réinitialiser les fichiers de configuration de nginx ». Les objectifs opérationnels et pédagogiques de ce TP sont décrits en bas de la page.

Introduction aux « hôtes virtuels »

Lorsqu'on entre une URL `http://domaine/page.html` dans un client web (un navigateur graphique comme `firefox` ou le logiciel `wget` par exemple), le nom de domaine de l'URL est utilisé deux fois :

- une première fois lors de la résolution de nom (DNS) qui permet de connaître l'adresse IP du système qui héberge le serveur web qui sert la page web correspondant à l'URL.
- une seconde fois, lors de l'échange HTTP avec le serveur web, afin d'identifier le site web dont vous demandez la page web.

En effet, un serveur web peut servir plusieurs sites web, ainsi `/page.html` n'est pas suffisant pour déterminer une page web fournie par un serveur web, car selon le nom de domaine, il peut s'agir de pages différentes.

Par exemple, si vous visitez les sites `https://rdos.lipn.univ-paris13.fr/`, `https://ask.sagemath.org/` et `https://sysadmin2526.netlib.re/` vous tombez sur des sites web différentes, alors que c'est le même serveur web qui les sert, et à chaque fois, vous avez demandé la page qui se trouve à la racine du site :

```
$ dig +short A rdos.lipn.univ-paris13.fr
194.254.163.53
$ dig +short A ask.sagemath.org
194.254.163.53
$ dig +short A sysadmin2526.netlib.re
194.254.163.53
```

Par ailleurs, souvenez-vous de la partie réseau du cours (et du cours de système et réseau), lorsqu'on simule un client web avec `socat` :

```
$ socat STDIO TCP:example.com:80,crlf << EOF
GET /index.html HTTP/1.1
Host: example.com

EOF
```

le nom de domaine `example.com` apparaît une première fois pour établir la connexion TCP et une seconde fois lors de l'échange avec le serveur web pour renseigner la valeur du champ `Host`, qui est obligatoire dans le protocole HTTP.

On peut connaître l'adresse IPv4 du système dont le nom de domaine est `example.com` :

```
$ dig +short A example.com
23.215.0.136
```

Si on remplace `example.com` par son adresse IPv4 dans la commande `socat`, ça va encore marcher :

```
$ socat STDIO TCP:23.215.0.136:80,crlf << EOF
```

```
GET /index.html HTTP/1.1
Host: example.com
```

EOF

Par contre, si on remplace **example.com** par son adresse IPv4 dans l'échange HTTP, le serveur ne va pas savoir de quel site il doit servir la page `/index.html` :

```
$ socat STDIO TCP:23.215.0.136:80,crlf << EOF
GET /index.html HTTP/1.1
Host: 23.215.0.136
```

EOF

La technique permettant à un même serveur web de servir des sites web différents selon le nom de domaine (donnant l'illusion qu'il s'agit de systèmes différents) s'appelle *name based virtual host* (fr: hôte virtuel basé sur le nom), avec les dérivés *virtualhost*, *vhost*, etc.

L'objectif (opérationnel) de ce TP est de vous faire héberger plusieurs sites web sur votre conteneur. C'est aussi une étape nécessaire pour le chiffrement des connexions entre le client et le serveur web à l'aide d'un certificat TLS (anciennement SSL), car celui-ci est lié à un ou plusieurs noms de domaine.

Préparation du premier site web (site perso à propos de vous-même)

On suppose que vous avez enregistré le nom de domaine **exemple.netlib.re** et que son enregistrement AAAA pointe vers l'adresse IPv6 de votre conteneur (cf TP précédent sur le DNS). Dans toute la suite du TP, vous remplacerez **exemple.netlib.re** par le nom de domaine que vous avez enregistré pendant le TP sur le DNS.

1. Assurez-vous que l'URL `http://exemple.netlib.re` affiche bien la page web que vous avez écrite dans `/var/www/html/index.html` lors du TP sur la découverte de **nginx**. Si ça n'est pas le cas, vous avez mal configuré votre nom de domaine, rendez-vous sur le TP « enregistrer un nom de domaine pour votre conteneur ».

Le répertoire qui accueillera le contenu de vos sites web sera `/srv` (voir **man 7 hier**).

2. Créez un répertoire nommé **exemple.netlib.re** dans `/srv/` pour accueillir le contenu de votre site web principal.
3. Dans ce répertoire, créez une page web d'accueil nommée `index.html` qui parle de vous. Vous pouvez par exemple commencer un portfolio avec votre CV, vos réalisations (liens vers les dépôts git de vos projets de programmation, etc). Vous pouvez bien sûr commencer par une page web minimaliste. Vous pouvez aussi rester anonyme.

Configuration du serveur web

Lors du TP sur l'hébergement d'une page web, vous avez observé le fonctionnement du serveur web **nginx**, sans modifier ses fichiers de configuration.

Le fichier de configuration par défaut, `/etc/nginx/sites-available/default` contient la ligne :

```
server_name _;
```

Le underscore `_` signifie que ce bloc **server** s'applique à tous les noms de domaine qui n'auraient pas été définis ailleurs.

Ce fichier est fourni par le paquet **nginx-common** qui est une dépendance du paquet **nginx** installé lors d'un TP précédent. Ainsi, si vous modifiez ce fichier, il pourra être écrasé lors d'une mise à jour du paquet **nginx-common**. Il est donc recommandé de ne jamais modifier ce fichier, mais plutôt d'en créer d'autres.

Ce fichier a un rôle de configuration par défaut, mais aussi un rôle de documentation par l'exemple. Dans le bloc de configuration principal, on peut voir des exemples (commentés) de configuration de SSL ou d'interprétation de scripts PHP.

Le dernier paragraphe de ce fichier (tout en bas) donne un exemple de configuration autonome pour un virtual host.

4. Lisez les 2 lignes de commentaires au début de ce paragraphe.
5. Copiez ce paragraphe dans un fichier du nom de votre domaine principal `exemple.netlib.re` et décommentez la partie qui n'est pas du commentaire.

Astuce : si vous utilisez `vim`, plutôt que sélectionner le dernier paragraphe à la souris et supprimer les `#` un par un à la main :

- copiez l'intégralité du fichier `default` sous le nom `exemple.netlib.re` avec la commande `cp`.
- éditez le fichier `exemple.netlib.re` avec `vim` et :
 - allez à la fin du fichier avec `G`
 - remontez d'un paragraphe avec `{`
 - supprimez tout ce qui se trouve entre le curseur et le début du fichier avec `dgg` (`d` pour `delete` et `gg` pour le début du fichier)
 - descendez de 5 lignes avec `5j` ou `5↓`
 - passez en mode « visuel bloc » avec `CTRL+v`
 - descendez jusqu'à la fin du fichier avec `G`, le mode visuel bloc sélectionne toute la première colonne.
 - supprimez la partie sélectionnée avec `x`
 - sauvegardez avec `:w`
- envisagez de quitter `nano`

6. Adaptez le fichier à votre situation. En particulier, modifiez la valeur des variables `server_name` et `root`.
7. Passez ce site web en production :
 - i. activez (en: enable) cette configuration à l'aide d'un lien symbolique depuis le répertoire `/etc/nginx/sites-enabled/`. Vous pouvez relire les questions 7, 8 et 9 du TP « Héberger une page web ».
 - ii. rechargez la configuration de `nginx`. Vous pouvez lire les commandes de « gestion des services » sur la feuille des commandes du cours.

Conseil : avant de recharger la configuration de `nginx` avec `systemctl`, vous pouvez vérifier que vous n'avez pas fait d'erreur de syntaxe en testant votre configuration :

```
# nginx -t
```

8. Vérifiez que, lorsque vous consultez l'URL `http://exemple.netlib.re`, vous obtenez bien la page de votre nouveau site personnel (pas la page que vous aviez faite lors du TP de découverte de `nginx`).

Second site web (sur un sujet de votre choix)

9. Choisissez un sujet de votre choix, par exemple une de vos passions, ou un sujet qui vous tient à coeur que vous souhaiteriez explorer ou partager.
10. Choisissez un sous-nom de domaine correspondant à ce sujet, du type `<sujet>.exemple.netlib.re`.
11. Configurez votre fichier de zone pour que le nom de domaine `<sujet>.exemple.netlib.re` pointe aussi vers l'adresse IPv6 de votre conteneur. Pour alléger la configuration du DNS, au lieu de recopier l'IPv6 dans l'enregistrement `AAAA` pour ce sous-domaine, utilisez l'enregistrement `CNAME`. Vous pouvez utiliser des chemins relatifs au domaine principal (l'*apex* de la zone), et désigner celui-ci par `@`.

12. Comme pour votre site principal, créez un répertoire pour le contenu de votre site dans `/srv` et un fichier de configuration pour `nginx` correspondant au site `< sujet>.exemple.netlib.re`.

Votre second site web doit contenir au moins deux pages web. Toutes les pages web de votre second site web doivent être accessibles depuis la page d'accueil de ce second site web.

Rappel : en HTML pour faire un hyperlien, on utilise une balise de type `<a>` (a comme fr:ancree, en:anchor) :

```
<a href="URL_DU_LIEN">TEXTE_DU_LIEN</a>
```

13. Passez votre second site web en production et assurez vous que lorsque vous consultez l'URL `http://< sujet>.exemple.netlib.re` depuis un navigateur web, c'est bien ce second site qui apparait.
14. Modifiez la page d'accueil de votre site principal (dont le domaine correspond à l'apex de votre zone), et faites en sorte que celle-ci ait un hyperlien vers votre second site.

Désactivation de la configuration par défaut

15. Désactivez, sans la supprimer, la configuration `default` fournie par la distribution.
16. Rechargez la configuration de `nginx`.
17. Vos deux sites web doivent continuer à fonctionner.

Conclusion

18. Ajoutez les URL de vos deux sites web (personnel et sujet/passion) sur votre page personnelle du wiki.

Lorsque vous avez vos deux sites complètement fonctionnels, que la configuration par défaut est désactivée et que les URL de vos sites sont sur votre page wiki, vous pouvez ajouter le tag `web.vhost` à votre page wiki.

Objectifs du TP :

- objectifs opérationnels du TP :
 - faire héberger plusieurs sites web indépendants par un même serveur web.
 - prendre le contrôle de la configuration de `nginx` et préparer le terrain pour le chiffrement TLS.
- objectifs pédagogiques du TP :
 - configurer un service en modifiant un template existant.
 - comprendre comment l'utilisation de liens symboliques permet d'activer/désactiver une configuration.
 - découvrir la gestion des services avec `systemctl`